



Recursion Trees

Draw the recursion, cost each level, and ask one question about how the levels change

Session 10 of 24 · Tuesday 20 October 2026 · 2:30 – 3:50 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 10 OF 24

Recursion Trees

Tuesday, 20 October 2026 · 2:30 – 3:50 PM

LEARNING OBJECTIVES

- ✓ Build a recursion tree from a recurrence in four steps.
- ✓ Compute the cost of one level and count the number of levels.
- ✓ Decide which of the three regimes applies: root-dominated, balanced, or leaf-dominated.
- ✓ Solve $T(n) = 2T(n/2) + n$, $T(n) = 2T(n/2) + 1$ and $T(n) = 3T(n/2) + n$ by tree.
- ✓ Say what the Master Theorem is for, and why we are not using it.

FROM SESSION 9

Unrolling works beautifully when there is **one** recursive call. With two, the expression branches at every substitution and the algebra becomes unmanageable — as Exercise 4 was designed to show you. A picture handles branching far better than a line of algebra does.

1 • The Method

12 min

1

Draw the root and label it with the *non-recursive* work the top-level call does: $f(n)$. Not $T(n)$ — just the

glue.

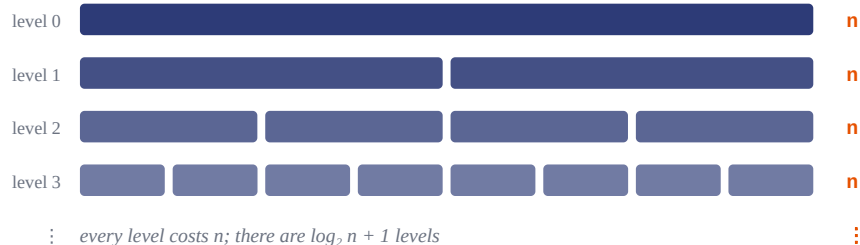
- 2 **Give it a children**, each a subproblem of size n/b , and label each with the glue *it* does: $f(n/b)$. Repeat downward.
- 3 **Cost each level**: multiply the number of nodes at that level by the work per node.
- 4 **Count the levels**, then add the level totals.

Why this is an argument and not merely a drawing

Every node of the tree corresponds to exactly one invocation of the function, and its label is exactly the work that invocation does outside its own recursive calls. So the sum of all the labels is the total work — no invocation is counted twice and none is missed. Grouping that sum by level is just a convenient order of addition.

2 • The Central Case: $T(n) = 2T(n/2) + n$ 18 min

$$T(n) = 2T(n/2) + n$$



Level by level. Level 0 is one problem of size n , doing n units of glue. Level 1 is two problems of size $n/2$, doing $n/2$ each — total n again. Level 2 is four problems of size $n/4$, doing $n/4$ each — total n . The width of the bars shrinks but the number of bars grows at exactly the same rate, so every level costs the same.

$$\text{cost of one level} = n \cdot \text{number of levels} = \log_2 n + 1$$

$$T(n) = n \cdot (\log_2 n + 1) = \Theta(n \log n)$$

Where does $\log_2 n + 1$ come from? The subproblem size is $n, n/2, n/4, \dots$, and the recursion stops when it reaches 1. That happens after $\log_2 n$ halvings — exactly the count we derived for binary search in Session 9, and exactly Chantal's sixteen questions from Session 1. The same quantity keeps reappearing because it is the same phenomenon.

3 · When the Levels Are Not Equal 25 min

3.1 $T(n) = 2T(n/2) + 1$ — the leaves win

Same split, but each node does only constant work. Level d holds 2^d nodes costing 1 each, so level d costs 2^d . The level costs are 1, 2, 4, 8, ... — they **double** as we descend.

$$1 + 2 + 4 + \dots + n = 2n - 1 = \Theta(n)$$

This is the doubling series from Session 5, and there we noted that such a series is dominated by its *last* term. The bottom level alone holds n nodes and accounts for half the total. So the answer is simply **the number of leaves**.

WHY BINARY TREE TRAVERSAL IS $\Theta(N)$

Visiting every node of a balanced binary tree with n nodes has exactly this shape: two recursive calls, constant work at each node. Students often guess $\Theta(n \log n)$ because “there are $\log n$ levels”. There are — but the levels are not equal, and the last one alone contains half the tree.

3.2 $T(n) = 2T(n/2) + n^2$ — the root wins

Level 0 costs n^2 . Level 1 is two problems of size $n/2$, each doing $(n/2)^2 = n^2/4$, so the level costs $n^2/2$. Level 2 costs $n^2/4$. The level costs **halve** as we descend:

$$n^2 + n^2/2 + n^2/4 + \dots < 2n^2 = \Theta(n^2)$$

The halving series from Session 5 is dominated by its *first* term. The root does more work than everything below it put together, so the answer is just $f(n)$.

3.3 $T(n) = 3T(n/2) + n$ — the leaves win again, less obviously

Three subproblems, each of half the size. Level d holds 3^d nodes of size $n/2^d$, so it costs $3^d \cdot n/2^d = n(3/2)^d$. The level costs are n , $1.5n$, $2.25n$, ... — growing by a factor of $3/2$, so again the bottom dominates. The number of leaves is $3^{\log_2 n} = n^{\log_2 3}$.

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

This is the shape of Karatsuba's fast multiplication algorithm: three half-sized multiplications instead of the four that schoolbook multiplication needs. Reducing a from 4 to 3 moved the exponent from 2 to 1.585 — which is why that trick was worth a paper.

3.4 The three regimes on one page

Recurrence	Level costs going down	Who dominates	Answer
$T(n) = T(n/2) + 1$	1, 1, 1, ...	balanced	$\Theta(\log n)$
$T(n) = 2T(n/2) + n$	n, n, n, ...	balanced	$\Theta(n \log n)$
$T(n) = 2T(n/2) + 1$	1, 2, 4, ... (grow)	the leaves	$\Theta(n)$
$T(n) = 3T(n/2) + n$	n, 1.5n, 2.25n, ... (grow)	the leaves	$\Theta(n^{1.585})$
$T(n) = 2T(n/2) + n^2$	$n^2, n^2/2, \dots$ (shrink)	the root	$\Theta(n^2)$

ONE QUESTION SETTLES ALL OF THEM

Going down one level, does the total work grow, stay the same, or shrink?

Grows → the bottom dominates; the answer is the number of leaves, $a^{\log_b n} = n^{\log_b a}$.

Stays → multiply one level's cost by the number of levels: $f(n) \cdot \log n$.

Shrinks → the root dominates; the answer is $f(n)$.

Memorise this question, not the five rows above it. The rows are consequences.

On the Master Theorem

There is a general theorem covering $T(n) = aT(n/b) + f(n)$, which compares $f(n)$ against $n^{\log_b a}$ and produces exactly the three regimes above. It is stated in **Appendix A** at the end of this lecture series, together with the awkward cases it handles and the gap where even it does not apply. We do not use it in the main line of the course. A rule that returns an answer without a picture is a poor way to *learn* where the answer comes from, and every recurrence you will meet this term yields to the question in the box above. Once you can draw the tree in your sleep, read the appendix and the theorem will look obvious — which is the right order.

4 • A Tree That Is Not Balanced 15 min

Not every recursion splits evenly. Consider $T(n) = T(n/3) + T(2n/3) + n$ — the shape of quicksort when the pivot always lands one third of the way along, which we will meet on 3 November.

Each level still costs n , because the two children's sizes add up to the parent's: $n/3 + 2n/3 = n$. That holds all the way down. But the branches now bottom out at different depths: the leftmost path shrinks by a factor of 3 each step and ends after $\log_3 n$ levels, while the rightmost shrinks by only $3/2$ and survives for $\log_{3/2} n$ levels — about 1.7 times longer.

$$n \cdot \log_3 n \leq T(n) \leq n \cdot \log_{3/2} n$$

both bounds are $\Theta(n \log n)$, since logarithm bases differ only by a constant

WHY THIS MATTERS MORE THAN IT LOOKS

A wildly unbalanced-looking split — one third against two thirds — gives the **same complexity class** as a perfect halving. Only the constant changes. This is the reason quicksort works in practice despite almost never splitting evenly, and it is worth carrying into November.

SESSION 10 TAKEAWAY

Draw the tree, cost one level, count the levels, and ask whether the level costs grow, hold, or shrink going down. Grow $\rightarrow$ count the leaves. Hold $\rightarrow$ one level times the number of levels. Shrink $\rightarrow$ the root. Three regimes, one question, no theorem required. The recurrences of this course are now solved problems, and on Friday we finally meet the algorithm that $T(n) = 2T(n/2) + n$ was written for.

IN-CLASS EXERCISES (15 min — work in pairs)

1. Draw the tree and give Θ for each: (a) $T(n) = 4T(n/2) + n$; (b) $T(n) = 2T(n/4) + n$; (c) $T(n) = T(n/2) + n$; (d) $T(n) = 9T(n/3) + n^2$.
2. For each of the four above, first predict the regime (grow / hold / shrink) *before* computing, then check yourself.
3. Two students disagree about $T(n) = 2T(n/2) + n$. One says $\Theta(n \log n)$; the other says the tree has n leaves so it must be $\Theta(n)$. Locate the error in the second argument precisely.
4. For $T(n) = aT(n/2) + n$, at what value of a does the answer stop being $\Theta(n \log n)$? What happens just below that value, and just above?
5. Draw the tree for $T(n) = T(n/5) + T(4n/5) + n$ and bound the number of levels on the shortest and longest paths.

Homework • due Friday 23 October

1. For each recurrence, draw the tree, state the level costs, name the regime, and give Θ : $T(n) = 2T(n/2) + n^2$; $T(n) = 4T(n/2) + n$; $T(n) = 8T(n/2) + n^2$; $T(n) = 3T(n/3) + n$; $T(n) = 2T(n/2) + n \log n$.
2. Write a Python function that takes a , b and a description of f , and prints the cost of the first six levels of the corresponding tree. Use it to check three of your answers above numerically.

3. Verify two of your answers from question 1 by substitution.
4. For $T(n) = T(n/3) + T(2n/3) + n$, compute the exact total work at levels 0, 1 and 2 for $n = 81$, and confirm each level costs 81.
5. Reading: CLRS §4.4 (recursion trees). Then read §4.5 (the Master Method) *only* as far as the statement of the theorem, and write one paragraph matching its three cases to our three regimes.

Next session · Friday 23 October: Merge sort. The algorithm behind $T(n) = 2T(n/2) + n$ — built, proved, raced against insertion sort on real data, and then extended to count inversions for free. It is also the last new material before Workshop II and the midterm.